

Executive Summary

A **university timetable management system** is a complex software application that goes far beyond “put classes into slots.” It requires modeling courses, classes, teachers, rooms, student groups and their constraints, then generating, validating, and publishing schedules under real institutional rules. We surveyed the literature (e.g. ITC-2007 Curriculum-based Course Timetabling ¹, University timetabling surveys ² ³, and current solutions like UniTime) and identified the key prerequisites:

- **Scope Definition:** Decide the problem variant. For example, schedule one department’s semester vs. the whole university; use fixed student “curricula” (course groups) vs. actual enrollments; include only lectures or also labs and tutorials; schedule rooms as well as times. Each choice alters the model. For instance, ITC-2007 Track 3 (curriculum-based) assumes a fixed curriculum structure and only weekly lectures ¹ ⁴. Unspecified choices (e.g. whole campus vs. one faculty) should be noted, with options given (e.g. “initially model a single faculty, extend to multi-faculty later”).
- **Stakeholders:** At minimum, include *Timetable Officers (Admins)* who enter data and finalize schedules, *Teachers* (provide availability, view and request changes), *Students* or *Student Representatives* (view timetables, report clashes), *Department Heads* (set policies), and *Room/Facility Managers*. Each has data inputs (e.g. availability constraints, room feature needs) and output needs (e.g. personalized calendars). The research shows that addressing lecturer and student needs (e.g. preferred times, clarity) is crucial for acceptance ⁵ ⁶. Common pain points include last-minute changes, hidden conflicts, and non-user-friendly interfaces.
- **Workflow:** The system should follow a realistic workflow. Data entry/validation comes first (import course lists, teacher assignments, student groups, room data, holidays, etc.), then a solver generates one or more candidate schedules, which are reviewed for conflicts. Administrators then manually adjust problematic classes and re-run the solver or finalize the result, before publishing the timetable to users ⁷ ⁸. In practice, commercial systems loop this (adjust, re-optimize) until all stakeholders are satisfied. We will model this lifecycle in the app (e.g. “Input Data → Generate Timetable → Show Conflicts → Manual Edit → Publish/Version”).
- **Domain Model:** Based on benchmarks and products (UniTime, OptaPlanner), core entities are: *Course* (with number of sessions, student count, department), *Class/Lecture* (a single meeting of a course), *Teacher*, *Room* (with capacity, features), *Timeslot* (day+period), *Student Group/Curriculum*, *Availability/Preference*, and *Schedule* with *Assignments*. Each Class is taught by one Teacher and attended by one or more Student Groups; it is assigned to one Room and one Timeslot in the final schedule. (See ER diagram below.) We will treat **each lecture/tutorial session as a separate planning entity**, not the entire course as one block ¹ ⁹.

erDiagram

```
COURSE ||--o{ CLASS : "consists of"
TEACHER ||--o{ CLASS : "teaches"
ROOM ||--o{ CLASS : "hosts"
CLASS }o--|| TIMESLOT : "scheduled in"
```

```
STUDENTGROUP ||--o{ CLASS : "attends"
CURRICULUM ||--o{ COURSE : "includes"
```

- **Constraints:** We must distinguish **hard constraints** (feasibility) from **soft constraints** (quality). From the ITC-2007 model and UniTime, typical hard constraints include:
 - **Teacher conflict:** No teacher can have two classes at once ¹⁰.
 - **Room conflict:** No room can hold two classes simultaneously ¹¹.
 - **Student group/curriculum conflict:** A student or curriculum cannot attend two classes at the same time ¹².
 - **Class coverage:** Every required session of each course must be scheduled.
 - **Room suitability:** A room must have required features (e.g. labs, projectors) for the class.
 - **Room capacity:** Capacity $\geq$ enrolled students (often treated as hard) ¹³.
 - **Availability:** Classes cannot be placed in periods blocked for the teacher or institution ¹⁴.
 - **Holiday blocks:** (Specify global blocked times, e.g. no classes on holidays).

Soft constraints measure desirability. Standard ones from ITC and UniTime include:

- **Room capacity penalty:** minimize exceeding student count ¹³.
- **Minimum working days:** Spread a course's lectures over at least a given number of days ¹⁵.
- **Curriculum compactness:** Minimize isolated lectures for each student group (prefer back-to-back) ¹⁶.
- **Room stability:** Prefer to use the same room for all lectures of a course ¹⁷.
- **Teacher preferences:** Honor preferred/unpreferred time slots for teachers (often soft).
- **Late hours:** Avoid very late-afternoon or evening classes for student/teacher comfort.
- **Consecutive scheduling:** (User suggestion) Prefer grouping classes to reduce idle gaps (saves energy/heating) or build continuous sessions.
- **Traffic-balancing rotation:** (User) Alternate heavy classes/days among groups to reduce travel congestion.
- **Morning-weighting:** (User) Favor morning slots in hot climates or for "freshness."
- **Day-end cutoff:** (User) Aim to finish schedules by a set time to reduce fatigue.
- **Room proximity:** (User) Prefer that sequential classes for the same group are in nearby rooms.
- **Course grouping options:** (User) Some courses may need non-consecutive placement by policy.

We will tabulate the main constraints and their type (excerpt below):

Constraint	Type	Description (hard=H, soft=S)
Teacher conflict	Hard ¹⁰	A teacher cannot teach two classes at the same period.
Student conflict	Hard ¹²	A curriculum or student group cannot have two classes simultaneously.
Room occupancy	Hard ¹¹	No two classes can share a room at the same period.
Teacher availability	Hard ¹⁴	Respect teacher's unavailable periods (no assignment allowed).
Holiday block	Hard	No classes scheduled on official holidays or break periods.
Room capacity	Hard/Soft ¹³	Enforce or penalize if class size exceeds room capacity.

Constraint	Type	Description (hard=H, soft=S)
Required sessions	Hard	All planned sessions for each course must be scheduled.
Room features	Hard	A class requiring equipment (e.g. lab) must use a room with it.
Minimum working days	Soft ¹⁵	Spread each course's lectures over at least N days.
Curriculum compactness	Soft ¹⁶	Minimize isolated lectures per curriculum (pref. consecutive).
Room stability	Soft ¹⁷	Prefer same room for all sessions of a course.
Teacher prefs (slots)	Soft	Reward placement of classes in teacher's preferred slots.
Time-of-day prefs	Soft	(e.g. avoid late slots; prefer mornings) for quality.
Consecutive classes	Soft	Favor back-to-back scheduling (reduces gaps/energy use).
Travel/traffic	Soft	(e.g. rotate groups/days) to balance student commute load.
Room proximity	Soft	Try to schedule sequential classes of a group in nearby rooms.

(*HC=hard constraint*, *SC=soft*) These will be formally defined in the system (with exact penalty values), but the principle is that hard constraints must never be broken, and the solver will minimize weighted soft-constraint penalties ³ ¹⁸.

- **Data Requirements & Validation:** A robust TMS needs rich input data. We will require:
- **Academic calendar:** Number of teaching days per week, number of slots each day.
- **Departments/Programs** and **Student Groups/Curricula** (defining which students attend which courses).
- **Courses:** Course code, name, number of lectures/labs per week, department, enrolled student count, assigned teacher, and the “minimum working days” for each course (spread requirement).
- **Lectures/Sessions:** Derived from courses (each course may have multiple identical weekly sessions).
- **Teachers:** List of teachers with their availability matrix (which slots they *cannot* teach) and any preferences.
- **Rooms:** Each room's name/ID, capacity, and features (lab, projector, etc.), plus any room-specific availability blocks.
- **Global Constraints:** Lists of holiday dates or blocked periods (e.g. no classes on Friday afternoon).
- **Optional Preferences:** e.g. teacher preferred slots, hard/easy hours, class grouping options.

Input validation must catch missing or inconsistent data (e.g. a course without a teacher, zero sessions, or a room assigned beyond capacity). We will normalize data in relational tables (see Database Schema section) and validate via referential integrity.

Sample data formats: For example, a JSON or CSV schema might include:

CourseID,Name,Dept,TeacherID,Enrolled,Sessions,MinDays
 1,Calculus,Math,10,120,3,3
 2,Physics,Phys,11,80,2,2

TeacherID,Name
 10,Dr. Alice
 11,Prof. Bob

RoomID,Name,Capacity,Features
 100,RM101,150,
 101,LabA,40,lab

CurriculumID,Name
 1,CS-Year1

CurriculumID,CourseID
 1,1
 1,2

Unavailable (Entity,ID,Day,Slot)
 Teacher,10,3,4 ; (Teacher 10 cannot teach Wed slot4)

We will prepare **three test tiers** for development and evaluation: small (e.g. ~5–10 classes, ~3 teachers, 3 rooms), medium (e.g. 30–50 classes across one department), and large (100+ classes, 30+ teachers, dozens of rooms across a faculty or university level). Real benchmarks (ITC instances) often have 100–400 lectures ¹⁹, so we aim similarly for large-scale testing.

- **Functional Requirements:** In an SRS style, the core functions will include:
- **Authentication & Roles:** Login system with roles (Admin/Timetable Officer, Teacher, Student).
- **Master Data Management (CRUD):** Manage departments, courses, classes/sessions, teachers, rooms, curricula/student groups, and timeslots.
- **Availability & Preferences Input:** Enter teacher unavailable slots, room unavailability, and preference data (rooms or times teachers prefer or avoid).
- **Constraint Configuration:** Define and weight soft constraints (e.g. preference importance).
- **Schedule Generation:** Trigger solver execution to generate a timetable (could be a button “Generate”).
- **Conflict/Violation Reporting:** After generation, display any hard constraint violations (ideally none in a correct solver) and soft-constraint penalty breakdown.
- **Timetable Views:** Visualize the schedule in grid form by teacher, by room, and by student group (week-by-week).
- **Filtering & Export:** Filter timetable by selected course/teacher/class or export to PDF/CSV.
- **Manual Adjustment:** Allow an admin to manually move a class (drag/drop) to resolve conflicts, with real-time revalidation.
- **Publish & Notifications:** Mark a schedule as “published” and optionally send notifications to users; maintain versions of schedules.
- **Logging/Audit:** Keep logs of generation runs and changes for audit/troubleshooting.

These match literature on TMS requirements ⁷ ⁸ . Additional specifics: students should access only their program's timetables; teachers should see only their assignments; admins see all.

- **Non-Functional Requirements:** Quality goals are critical. Based on timetabling research, we set:
- **Performance:** The system should generate a timetable for a medium-sized department (e.g. 100 lectures) within a target time (e.g. under 5 minutes initially, ideally much faster for smaller problems) ⁷ . Typical generation should be under 30 seconds for real-time use, with a limit (e.g. 5 min) on large jobs.
- **Responsiveness:** UI actions (e.g. viewing timetables, saving inputs) must respond under 1–2 seconds.
- **Scalability:** System should scale to 1,000+ courses/lectures (through efficient algorithms and possibly distributed solving). Adding more rooms/teachers shouldn't drastically degrade performance.
- **Accuracy & Reliability:** Schedules must obey hard constraints (no silent conflicts). Stored data must be safe (regular backups). The solving process should be reproducible given same input.
- **Security:** Role-based access to data; sanitize all inputs; use HTTPS; protect sensitive data (e.g. via encryption).
- **Usability:** Intuitive forms for entering courses/constraints, clear conflict reports, and easy navigation. (User requirements studies stress ease-of-use and trust ²⁰ .)
- **Maintainability:** Modular design separating solver from UI so constraints or algorithms can be updated without reworking the entire system.
- **Availability:** The web app should be hosted reliably (99.9% uptime target) to allow scheduling any time.
- **Explainability:** The system should clearly report why a schedule is good or bad (see "Validation & Reporting").

(Where measurable targets exist, they should be quantified, e.g. response <2s, uptime 99.9%, etc.)

- **System Architecture:** We recommend a classic multi-tier web architecture (see component diagram below). A **frontend** (e.g. React/Angular) communicates with a **backend API** (e.g. Java Spring Boot or Python Flask) over HTTPS. The backend connects to a **relational database** (e.g. PostgreSQL) for data and user management. The **solver** (the optimization engine) can be a library/module in the backend (e.g. OptaPlanner embedded or OR-Tools calls) or a separate service. For example, a REST API endpoint like `/generate-schedule` could trigger a solver job. Results and violations are stored back in the database. The architecture should allow the solver to run as a background job (possibly in a container or cloud compute instance) without blocking the web server.

Deployment Options: The app can be containerized (Docker) with components: web server, app server, database, and solver. It could run on-premises or in the cloud. Authentication may integrate with campus SSO. Interfaces/API endpoints will include e.g. GET/POST for data, a POST to start solving, and endpoints to retrieve schedule results.

flowchart LR

```
UI["Web Frontend (React/Vue)"] -->|REST API| API["Backend API (e.g. Spring Boot)"]
API --> DB["Database (PostgreSQL)"]
API --> Solver["Scheduling Engine (OptaPlanner/OR-Tools)"]
Solver -->|Stores results| DB
```

Auth["SSO/Auth Service"] --> API
 %% Deployments could also include SIS integration, logs, etc.

This separation (UI ↔ API ↔ DB, with a solver component) allows us to swap out the solving technology later. The OptaPlanner documentation similarly structures problems as an input model plus planning entities that the solver assigns ²¹ ²². The component diagram will be part of documentation.

- **Database Schema:** We will use normalized tables for master data, constraints, and results. Key tables (with sample columns) include:

Table	Columns (Type)	Description
teacher	teacher_id PK, name, email, department_id, ...	Instructors
room	room_id PK, name, capacity, features (array), ...	Rooms with capacities/features
course	course_id PK, code, name, department_id, sessions/week, min_working_days, ...	Courses offered
curriculum	curriculum_id PK, name, description	A student group or degree plan
curriculum_course	curriculum_id (FK), course_id (FK)	(Many-to-many: courses in each curriculum)
class_session	session_id PK, course_id (FK), type (lec/tutorial/lab), teacher_id (FK), student_count, ...	Each scheduled meeting of a course
timeslot	timeslot_id PK, day_of_week, period_of_day (or time)	All available periods
teacher_unavail	teacher_id (FK), timeslot_id (FK)	Teacher unavailable slots
room_unavail	room_id (FK), timeslot_id (FK)	Room blocked slots
schedule	schedule_id PK, name, created_at, status, score_hard, score_soft	A specific generated timetable
assignment	assignment_id PK, schedule_id (FK), session_id (FK), room_id (FK), timeslot_id (FK)	One class assignment in a schedule
violation	violation_id PK, schedule_id (FK), type, description, weight	Stores conflicts (if any)

Columns will include data types (e.g. INT, VARCHAR) and foreign keys. For instance, the `class_session` table links each session to its course and teacher. The `assignment` table stores the solved time/room for each session in a given schedule. The `violation` table records any hard/soft constraint breaches in that schedule. This schema aligns with typical timetabling data models (UniTime uses similar entities, separating input courses from solved classes) ²³ ²⁴.

- **UI Screens & User Flows:** The web UI should support the user journey. Key screens/wireframes include:
 - **Login Page** (role-based access).
 - **Dashboard** showing current/published schedules and recent activity.

- **Master Data Forms:** e.g. *Teachers, Rooms, Courses, Curricula, Timeslots*, with tables and forms to add/edit.
- **Availability/Preferences:** Forms or grids where teachers mark unavailable times; admins mark global blocked periods; define room features/groups; set weights for soft constraints.
- **Schedule Generation Page:** A page where the admin selects the academic term, clicks “Generate”, and sees a progress indicator.
- **Conflict Report:** After generation, show a list of hard constraint violations (if any) and soft constraint penalties. (For example, list “Teacher X double-booked in Timeslot Y” or “5 students over capacity in Room Z”.) This helps users diagnose issues.
- **Timetable Viewer:** The core output. Provide an interactive weekly grid (days vs. time slots) view of classes. There should be filters (by teacher, by curriculum, by room) and toggle of teacher/class/room modes. A typical grid view might look like an Excel-like schedule (e.g. as in many institutional timetable systems). (For example, see typical weekly schedules in [Academic Scheduler’s UI][30], or [UniTime’s room timetable viewer][9†L88-L95] as inspiration.) If possible, classes should be drag-and-droppable for quick edits (highlighting only conflict-free slots).
- **Publication & Export:** Screens to “publish” a version (locking it), and to export timetables (PDF/CSV) for individuals or groups (e.g. “print all teacher schedules”).
- **Manual Edit:** If allowed, an interface to select a class and move it (with the system rechecking constraints).
- **Settings/Administration:** Manage user accounts, roles, and system configuration.

Example visuals: A weekly grid might have Monday–Friday columns and 8am–6pm rows, with colored blocks for each lecture (showing course code and room). A conflict report might be a table listing conflicts. (While we won’t embed actual images here, the important point is that the UI must present data clearly and allow intuitive editing.)

- **Validation, Explainability & Reporting:** We must show why a schedule is correct or not. After solving, the system should:
- **Compute violations:** Programmatically check all hard constraints (teacher double-books, room overlaps, unscheduled classes) and soft constraint penalties using the database of assignments. We will use rule logic similar to the ITC validator ²⁵ .
- **Score breakdown:** Present a summary score (e.g. hard=0, soft=123 points) and breakdown per soft constraint (e.g. “Curriculum Compactness: 40 pts”). This mirrors how competition organizers evaluate solutions ²⁵ ²⁶ .
- **Conflict report:** List each hard violation (e.g. “Teacher A has 2 classes in Slot X”). Clicking a violation could highlight the affected classes in the timetable grid. Soft violations can be summarized more generally.
- **Audit logs:** For each schedule run, log the input stats (courses, constraints) and output scores, so we can compare different runs.

These features make the system transparent. (ITC even provided a solution validator that prints all violations ²⁵ ; we should implement a similar internal check.)

- **Evaluation Plan & Metrics:** To measure success, we will run systematic tests on our chosen datasets. Key metrics include:
- **Feasibility:** whether hard constraints are all satisfied (should be true for final results).
- **Hard Violations:** count of any unmet constraints (target = 0).
- **Soft-Constraint Score:** total penalty; lower is better. We may report individual soft constraint tallies.
- **Generation Time:** solver runtime on each test instance.
- **Scalability:** how solve-time and quality scale with problem size (small vs. medium vs. large).
- **Quality vs. Users’ edits:** time saved by automated solution vs. pure manual (if measurable).

- **User metrics:** maybe count manual adjustments needed, or user satisfaction surveys (if within scope).

We will conduct experiments by solving small/medium/large test cases multiple times (with different random seeds if using heuristics) and chart the results. For instance, we could plot **instances vs solve time** and vs **score**. A sample testing plan timeline might be:

```

timeline
  title Evaluation Timeline
  2026-05-01 : Prepare test data and scripts
  2026-05-08 : Run solver on small-instance dataset
  2026-05-15 : Analyze small-instance results
  2026-05-22 : Run solver on medium-instance dataset
  2026-05-29 : Analyze medium-instance results
  2026-06-05 : Run solver on large-instance dataset
  2026-06-12 : Analyze large-instance results

```

We'll also compare against known benchmarks to validate our solver's effectiveness.

- **Benchmark Datasets & Sample Data:** We will use established benchmarks for credibility. The main one is **ITC-2007 Curriculum-based Course Timetabling (Track 3)**, which provides 21 real instances (from the University of Udine) with curricula, courses, teachers, rooms, etc. (seven "early" and seven "late" instance sets, plus hidden test cases) ²⁷ ²⁸ . These have 24–130 lectures and 5–36 curricula each, and are widely used for evaluating algorithms. Additional data sources include:
 - **Socha's dataset (2010)** for curriculum-based timetabling (if available publicly).
 - Any departmental data we can generate (e.g. synthetic but realistic for our institution).
 - Possibly data from the University of Sydney or other public UCTT instances used in literature ²⁹ .

We will format sample data in JSON or CSV as illustrated above. We may include an example JSON file with a handful of entries as a downloadable sample (e.g. "sample_data.json") or embed a snippet in documentation.

- **Technical Stack Recommendations:** For the solver and tech stack, we have options:
- **Java + OptaPlanner (Timefold):** OptaPlanner is a mature open-source constraint solver for scheduling, with built-in support for timetabling and a declarative constraint API. It embeds in Java/Spring apps and can visualize constraints in Drools rules. It's well-documented and used in real timetabling (UniTime historically was Java). Pro: seamless Java integration and enterprise support. Con: learning curve for Drools constraint rules and possibly slower on huge instances than CP-SAT.
- **Python + Google OR-Tools (CP-SAT):** OR-Tools' CP-SAT solver is very powerful and often faster on large combinatorial problems. Python bindings are convenient, and OR-Tools can handle complex linear/boolean constraints. Pro: state-of-the-art performance and active development by Google. Con: less built-in modeling for scheduling patterns (you'd model more manually) and less out-of-the-box for multi-criteria weighting. OR-Tools does have a Python interface for scheduling (see [OR-Tools scheduling docs][39]).
- **Hybrid/Microservice:** E.g., a Python microservice with OR-Tools for solving, called by a Java backend or vice versa. This could combine pros of both. But adds overhead.

For frontend, a modern JS framework (React, Vue, or Angular) is recommended. Database: PostgreSQL (or similar) for relational data. We'll choose based on team expertise; for example, a Java/Spring Boot

backend + React frontend + PostgreSQL is a common stack, or Python/Flask + Vue + SQLite/MySQL. Crucially, the solver logic should be decoupled enough that switching from OptaPlanner to CP-SAT later is feasible. The Solver component should be behind an interface (a planning service) so we can experiment with different algorithms (heuristics, CP, MILP) without rewriting the whole app.

- **MVP Scope & Out-of-Scope:** A realistic **Minimum Viable Product** will support: one-semester, department-level scheduling of lectures/labs; core hard constraints and a few soft preferences (as listed above); data entry screens; automated generation and conflict report; and basic timetable views. Out of scope for now:
 - Detailed student-by-student enrollment (we'll use fixed groups/curricula instead).
 - Multi-campus or travel-time optimization.
 - Exam scheduling or sectioning (these are separate scheduling problems).
 - Real-time updates beyond periodic re-generation.
 - Custom user-defined constraint languages (we'll use code/rules).
 - Integration with live SIS/LMS (we'll import CSVs instead).
 - Mobile apps or signboards (though PDF export is planned).

Future work could add those. But phase 1 should firmly nail the above functional and non-functional requirements.

Implementation Checklist

Below is a high-level prioritized checklist of development milestones:

1. **Project setup:** Choose tech stack, initialize repository, set up CI/CD, deployable architecture (containers).
2. **Domain model & database:** Design ER schema, create database, implement ORM/entities.
3. **Master data CRUD:** Build UI and APIs for Courses, Teachers, Rooms, Curricula, Timeslots, etc.
4. **Constraint input:** Implement forms for teacher/room availability and preferences.
5. **Validation logic:** Code checks for input sanity and basic hard-constraint filtering (no teacher-less course, etc).
6. **Schedule entity:** Design `Schedule` and `Assignment` tables; implement creation of blank schedules.
7. **Integrate solver:** Embed a solver library (OptaPlanner or OR-Tools) and code to convert DB data to solver model.
8. **Generation endpoint:** Create backend endpoint `/generate` that runs the solver and saves results.
9. **Conflict detection:** After generation, run validation to detect any remaining hard/soft violations (per ITC rules ³) and save them.
10. **Timetable UI:** Implement timetable views (grid by day) for teachers/rooms/groups.
11. **Conflict report UI:** Display violations list and score breakdown.
12. **Manual editing:** Allow dragging a class to a new slot and recheck constraints in real time.
13. **Versioning/publish:** Allow saving named schedule versions, marking as "published", and disallow editing older versions.
14. **Export/printing:** Add CSV/PDF export for timetables and reports.
15. **User accounts & security:** Add login, roles, and access control.
16. **Performance tuning:** Benchmark with sample data, optimize solver config, maybe add caching.
17. **Documentation:** Write SRS, API docs, usage guide, plus diagrams (ER, architecture, user flows).
18. **Testing & validation:** Test on small/medium/large datasets, produce evaluation charts, fix issues.
19. **Review & polish:** UI improvements, error handling, finalize deployment.

With this comprehensive plan grounded in literature and real-world practice ⁷ ³, the timetable system will address both practical scheduling needs and serve as a solid software engineering project foundation.

Sources: Core ideas are drawn from the ITC-2007 timetabling definitions ¹ ³, UniTime documentation and case studies, published surveys ² ²⁷, and modern scheduling solution overviews ⁷ ¹⁸. These, along with the specified project requirements, inform the system design above.

¹ ¹⁰ ¹¹ ¹² ¹³ ¹⁴ ¹⁵ ¹⁶ ¹⁷ ¹⁹ ²⁶ **OptaPlanner User Guide**

https://docs.optaplanner.org/7.15.0.Final/optaplanner-docs/html_single/index.html

² ²⁸ ²⁹ **A Survey of University Course Timetabling Problem: Perspectives, Trends and Opportunities**

<https://www.graham-kendall.com/papers/chenetal2021a.pdf>

³ ⁴ ⁹ ²⁴ ²⁵ ²⁷ **icaps07-satellite.icaps-conference.org**

<https://icaps07-satellite.icaps-conference.org/workshop4/paper06.pdf>

⁵ ²⁰ **Microsoft Word - final**

<https://aircconline.com/ijsea/V7N3/7316ijsea01.pdf>

⁶ ⁷ ⁸ ¹⁸ **Research & Development**

<https://edtex.in/research-development>

²¹ ²² **A Guide to OptaPlanner | Baeldung**

<https://www.baeldung.com/opta-planner>

²³ **Course Timetabling Data Entry Manual**

<https://help.unitime.org/manuals/courses-entry>